

SOFTWARE REQUIREMENTS SPECIFICATION

for

SignVision

*An AI-based Real-time Sign Language Interpreter
with Smart-Home Integration*

Version 1.0

Prepared by:

- 1. Sidharth S Pillai (MUT22AD068)**
- 2. Shajudheen K (MUT22AD066)**
- 3. Ramnarayan R (MUT22AD058)**
- 4. Amaldev K U (MUT22AD017)**

Department of Artificial Intelligence and Data Science
Muthoot Institute of Technology and Science

December 6, 2025

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Document Conventions	2
1.3	Intended Audience and Reading Suggestions	2
1.4	Project Scope	2
1.5	References	3
2	Overall Description	3
2.1	Product Perspective	3
2.2	Product Functions	3
2.3	User Classes and Characteristics	4
2.4	Operating Environment	4
2.5	Design and Implementation Constraints	4
2.6	System Architecture	5
3	External Interface Requirements	6
3.1	User Interfaces	6
3.2	Hardware Interfaces	6
3.3	Software Interfaces	6
3.4	Communications Interfaces	7
4	System Features	7
4.1	System Feature 1: Dynamic Gesture Recognition	7
4.2	System Feature 2: Holistic Landmark Tracking	7
4.3	System Feature 3: Virtual Camera Broadcast	7
4.4	System Feature 4: Smart Home Automation (IoT Integration)	7
4.5	System Feature 5: Multilingual Text-to-Speech	8
5	Other Nonfunctional Requirements	8
5.1	Performance Requirements	8
5.2	Safety Requirements	8
5.3	Security Requirements	8
5.4	Software Quality Attributes	8
A	Glossary	9
B	Analysis Models	9
B.1	Data Flow Diagram	9
C	To Be Determined List	9

1 Introduction

1.1 Purpose

The purpose of this document is to comprehensively specify the software requirements for **SignVision**, an advanced assistive technology system designed to facilitate two-way communication between hearing/speech-impaired individuals and the general public.

The primary objective of the system is to translate dynamic Indian Sign Language (ISL) and American Sign Language (ASL) gestures into audible speech and text in real-time. This SRS document outlines the functional requirements, non-functional constraints, external interfaces, and system architecture necessary to develop a robust, low-latency translation engine that integrates seamlessly with modern video conferencing tools like Google Meet and Zoom.

1.2 Document Conventions

This document adheres to the IEEE 830-1998 standard for Software Requirements Specifications.

- The term "**System**" refers to the SignVision application.
- The term "**User**" refers to the signer (the person performing gestures).
- "**Inference Engine**" refers to the LSTM-based neural network backend.
- Typographical conventions include **Bold** for specific features or components and *Italics* for emphasis on technical terms.

1.3 Intended Audience and Reading Suggestions

- **Project Developers:** To understand the architectural flow, specifically the MediaPipe integration and LSTM model inputs/outputs.
- **Project Guides/Evaluators:** To verify that the proposed features align with the academic scope and objectives.
- **UI/UX Designers:** To understand the layout requirements for the dashboard and video overlay components.

It is recommended that developers focus primarily on Section 3 (Interfaces) and Section 4 (System Features), while project managers or guides review Section 1 and Section 2 for scope and feasibility.

1.4 Project Scope

SignVision is a desktop-based software solution that leverages Computer Vision and Deep Learning.

- **Core Functionality:** The application captures video input via a standard webcam, processes the frames to extract skeletal landmarks using MediaPipe Holistic, and classifies the temporal sequence of landmarks into meaningful words using a Recurrent Neural Network (LSTM).

- **Integration:** Unlike standalone translators, SignVision includes a Virtual Camera Broadcast Module. This allows the processed video feed—complete with text overlays—to be injected into third-party applications (Zoom, Teams, Google Meet) as a recognized video source.
- **Data Strategy:** The model is trained on a hybrid dataset comprising the standardized WLASL (World Level American Sign Language) dataset and custom-recorded gestures to ensure high accuracy for essential vocabulary.
- **Future Expansion:** The project architecture is modularized to support Phase 2 integration, which involves controlling IoT-enabled smart home devices (lights, fans) via specific sign language commands.

1.5 References

1. *MediaPipe Holistic: Simultaneous Face, Hand and Pose Prediction*, Google AI Developers, 2020.
2. *WLASL: A World-Level Dataset for Word-Level American Sign Language*, CVPR, 2020.
3. *Long Short-Term Memory (LSTM)*, Neural Computation, Sepp Hochreiter & Jürgen Schmidhuber, 1997.
4. *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998.

2 Overall Description

2.1 Product Perspective

SignVision operates as an intelligent middleware between the user's physical environment and digital communication platforms. It is designed as a standalone Python application.

- **Frontend:** A graphical user interface (GUI) built using libraries such as Tkinter or PyQt to display the camera feed, translation logs, and settings.
- **Backend:** The core logic resides in a Python environment utilizing TensorFlow/Keras for the deep learning inference and OpenCV for image processing.
- **Virtualization:** The system employs virtual device drivers (e.g., pyvirtualcam) to mimic a hardware webcam, allowing it to interface with any software that accepts video input.

2.2 Product Functions

The major functional capabilities of the system include:

1. **Video Acquisition & Preprocessing:** capturing live video streams and normalizing lighting/contrast for optimal tracking.
2. **Holistic Landmark Extraction:** simultaneously tracking 33 pose landmarks, 468 face landmarks, and 21 hand landmarks per hand.

3. **Sequence Recognition:** buffering landmark data over a sliding window of frames (e.g., 30 frames) to detect dynamic gestures rather than just static poses.
4. **Speech Synthesis (TTS):** converting the predicted text labels into audible speech using the Google Text-to-Speech (gTTS) API.
5. **Multilingual Translation:** offering the ability to translate the recognized English text into other languages (e.g., Malayalam, Hindi) via Google Translate API.
6. **Virtual Broadcast:** piping the augmented video feed to external conferencing tools.

2.3 User Classes and Characteristics

- **Administrator/Developer:** Has access to the model training scripts, dataset management tools, and configuration files to fine-tune hyperparameters.
- **End User (Signer):** The primary operator of the software. This user may have hearing or speech impairments and requires a simple, "one-click" interface to start translation.
- **Receiver (Non-Signer):** The individual on the other end of the video call who reads the subtitles or hears the synthesized speech. They do not interact with the software directly.

2.4 Operating Environment

The system is designed to operate in the following environment:

- **Operating System:** Windows 10/11 or Linux (Ubuntu 20.04+).
- **Processor:** Minimum Intel Core i5 or AMD Ryzen 5 (AVX instruction set support required for TensorFlow).
- **Memory:** Minimum 8GB RAM (16GB recommended for smooth multitasking).
- **GPU:** NVIDIA GPU with CUDA support is highly recommended to reduce inference latency, though the system is optimized for CPU inference via MediaPipe TFLite models.
- **Peripherals:** A standard USB webcam (720p or higher) and a microphone/speaker setup.

2.5 Design and Implementation Constraints

- **Lighting Sensitivity:** The accuracy of MediaPipe's landmark detection is heavily dependent on ambient lighting. The system must include threshold checks to warn users of poor lighting.
- **Vocabulary Limit:** The system can only recognize gestures on which it has been explicitly trained. It does not support "unseen" signs.
- **Computational Load:** Running video capture, AI inference, and virtual camera broadcasting simultaneously is resource-intensive. The code must be optimized to prevent thermal throttling on laptops.

2.6 System Architecture

The SignVision platform employs a modular pipeline architecture to ensure low-latency processing and scalability. The system is segregated into three distinct layers: Input/Presentation, Core Logic, and Output/Integration.

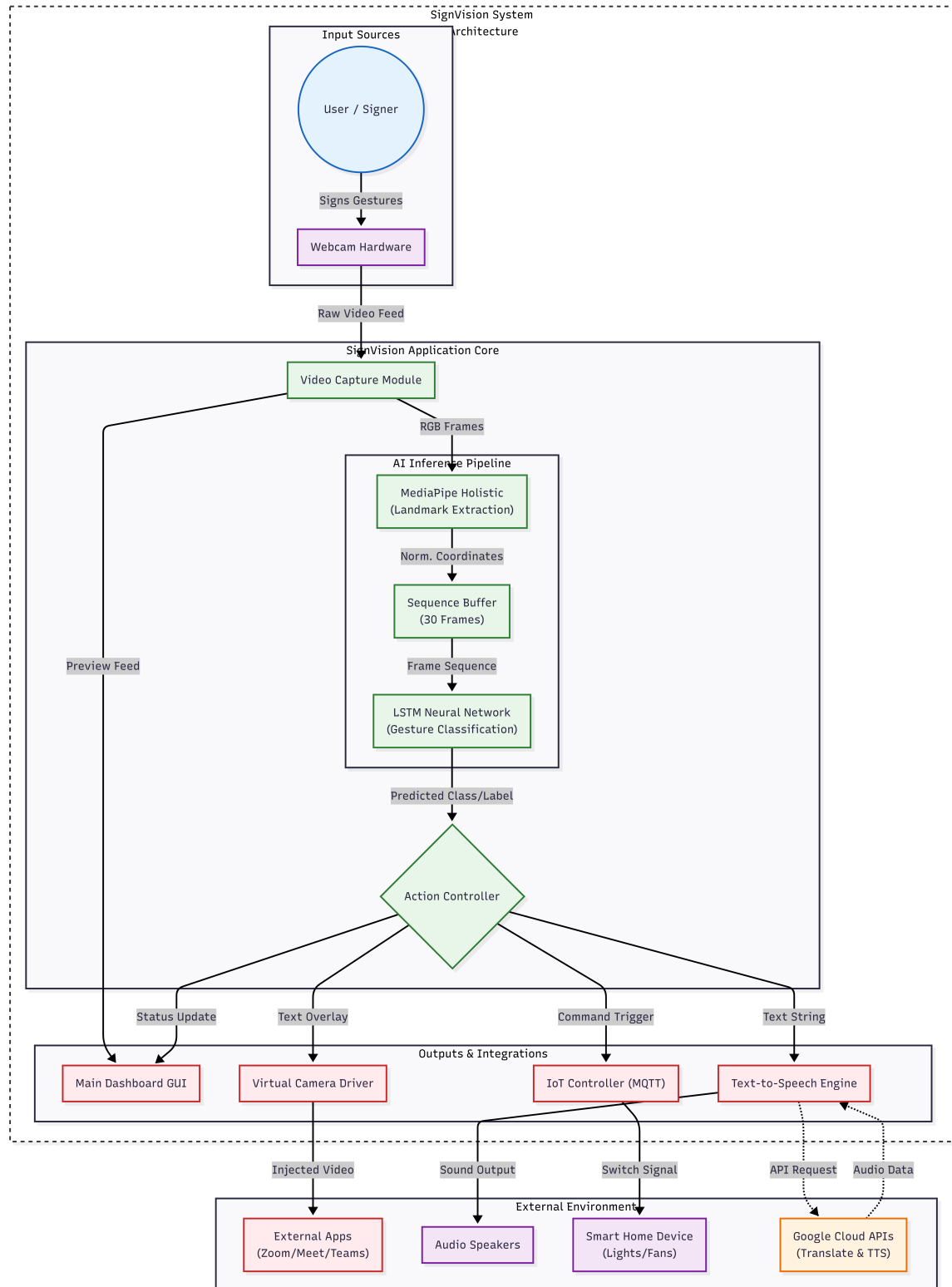


Figure 1: SignVision System Architecture

The data flows as follows:

1. **Input Layer:** Users interact with the hardware webcam.
2. **Core Processing:** The *Video Capture Module* feeds frames to the *MediaPipe* engine for landmark extraction. The resulting coordinates are buffered and analyzed by the *LSTM Inference Engine*.
3. **Output Layer:** The *Action Controller* routes the results to the GUI, the Text-to-Speech engine, the IoT module, or the Virtual Camera driver based on the recognized command.

3 External Interface Requirements

3.1 User Interfaces

The application will feature a minimalist Dashboard containing:

- **Live Preview Window:** A large central panel showing the webcam feed with skeletal overlays (wireframe hands/face) to confirm tracking.
- **Translation Box:** A text area located below the video feed that displays the current predicted sentence and a history of previous phrases.
- **Control Panel:** Buttons to Start/Stop the camera, toggle Audio (TTS), and calibrate the background.
- **Status Indicators:** Visual indicators for "System Ready," "Recording," and "Broadcasting."

3.2 Hardware Interfaces

- **Webcam Driver:** The system interfaces with the OS video drivers (DirectShow on Windows / V4L2 on Linux) to access the physical camera.
- **Audio Output:** The system connects to the default OS audio output device to play the synthesized speech.

3.3 Software Interfaces

- **MediaPipe API:** Google's open-source framework used for the initial landmark extraction pipeline.
- **TensorFlow/Keras:** The deep learning framework hosting the LSTM model.
- **OBS/PyVirtualCam:** Interface used to create a virtual video device that other applications can detect.
- **Google Cloud APIs:** Used for Text-to-Speech (gTTS) and language translation features.

3.4 Communications Interfaces

- **Internal Process Communication:** The system uses threading to manage the video loop separately from the GUI loop to prevent freezing.
- **Internet Protocol:** HTTP/HTTPS requests are generated when the user activates the Translation or TTS features that rely on cloud APIs.

4 System Features

4.1 System Feature 1: Dynamic Gesture Recognition

Description: The core capability of the system is to distinguish between static hand poses and dynamic gestures (movements over time).

Stimulus/Response: As the user performs a sign (e.g., waving "Hello"), the system captures a sequence of 30 frames. The model processes this sequence and outputs a probability distribution across all known classes. If the confidence score exceeds 85%, the label is accepted.

Functional Details: This feature utilizes a Long Short-Term Memory (LSTM) network. Unlike standard Convolutional Neural Networks (CNNs) that look at single images, the LSTM maintains a "memory" of previous frames, allowing it to understand the context of motion.

4.2 System Feature 2: Holistic Landmark Tracking

Description: The system visualizes the computer's understanding of the user's body language.

Stimulus/Response: When a user enters the frame, the system immediately superimposes a skeletal wireframe over their hands, face, and torso on the screen.

Functional Details: This is powered by MediaPipe Holistic. It extracts x, y, z coordinates for 543 distinct landmarks. These coordinates are normalized (relative to the user's position in the frame) to ensure the system works regardless of how close or far the user sits from the camera.

4.3 System Feature 3: Virtual Camera Broadcast

Description: This feature allows SignVision to "impersonate" a physical webcam.

Stimulus/Response: The user opens Zoom or Google Meet and selects "SignVision Camera" from the video settings. The conference call immediately displays the SignVision output (video + text overlay) instead of the raw webcam feed.

Functional Details: The system uses the 'pyvirtualcam' library to write processed frames into a virtual loopback device. This ensures compatibility with any application that accepts video input, without requiring plugins for those specific apps.

4.4 System Feature 4: Smart Home Automation (IoT Integration)

Description: An extension of the recognition engine to control physical devices.

Stimulus/Response: The user signs a specific command trigger, such as "Light On." The system recognizes this class and triggers an HTTP request to a local IoT server (e.g., NodeMCU/ESP8266).

Functional Details: The prediction engine maps the gesture label to a pre-configured URL endpoint. When the label "Light On" is detected with high confidence, a GET/POST request is sent to the smart switch IP address to toggle the relay.

4.5 System Feature 5: Multilingual Text-to-Speech

Description: Converts the recognized sign language into spoken audio in the user's preferred language.

Stimulus/Response: Upon recognizing a sentence, the system generates an audio file and plays it. If the user has selected "Malayalam" in settings, the English sign is translated first, then spoken.

Functional Details: The system utilizes the 'googletrans' library for text translation and 'gTTS' (Google Text-to-Speech) for audio synthesis.

5 Other Nonfunctional Requirements

5.1 Performance Requirements

- **Latency:** The end-to-end time from gesture completion to text display must be under 500 milliseconds to maintain conversational flow.
- **Frame Rate:** The output video stream must maintain a minimum of 24 Frames Per Second (FPS) to ensure the video does not look choppy to the receiver.
- **Accuracy:** The model should achieve a validation accuracy of at least 90% on the test dataset for the defined vocabulary.

5.2 Safety Requirements

- **Usage Disclaimer:** The software must display a disclaimer stating it is an assistive tool and not a certified interpreter for critical legal or medical situations where mistranslation could be dangerous.
- **Ergonomics:** The system should detect if the user has been signing for extended periods and suggest breaks to prevent repetitive strain injury (RSI).

5.3 Security Requirements

- **Data Privacy:** All image processing must occur locally on the user's machine. No video data should be uploaded to external cloud servers for inference.
- **API Security:** API keys used for Google Cloud services must be stored in encrypted environment variables, not hardcoded in the source files.

5.4 Software Quality Attributes

- **Reliability:** The system must recover automatically if the webcam is disconnected and reconnected.
- **Maintainability:** The codebase is modularized (Data Collection, Training, Inference) to allow for easy updates to the gesture vocabulary without rewriting the core application.
- **Portability:** Being Python-based, the software should be capable of running on Windows, macOS, and Linux with minimal changes to the environment configuration.

A Glossary

API Gateway A management tool that sits between a client and a collection of backend services.

LSTM (Long Short-Term Memory) A type of Recurrent Neural Network (RNN) capable of learning order dependence in sequence prediction problems.

MediaPipe A cross-platform framework for building multimodal applied machine learning pipelines, developed by Google.

WLASL World Level American Sign Language dataset, a large video dataset for sign language recognition.

IoT (Internet of Things) The network of physical objects embedded with sensors and software to connect and exchange data.

B Analysis Models

B.1 Data Flow Diagram

The data flow within SignVision follows a linear pipeline:

1. **Input:** Raw RGB frames from Webcam.
2. **Process A:** MediaPipe extracts 1662 numerical data points (Landmarks).
3. **Process B:** Data points are buffered into arrays of 30 frames.
4. **Process C:** The LSTM Model calculates probability across classes.
5. **Output:** The class with the highest probability is rendered as Text and Audio.

C To Be Determined List

1. **TBD1:** Exact list of vocabulary words for the final prototype.
2. **TBD2:** Final decision on using MQTT vs HTTP for IoT integration.
3. **TBD3:** Support for mobile device integration (Android App).